

Spring Core Maven

Exercise 1: Configuring a Basic Spring Application

Create a Maven Project

Project Name: LibraryManagement

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.library</groupId>
  <artifactId>LibraryManagement</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
  </properties>

  <dependencies>

    <!-- Spring Core -->
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-context</artifactId>
      <version>6.1.14</version>
    </dependency>

  </dependencies>

</project>
```

Configure the Application Context

Create the file:

```
src/main/resources/applicationContext.xml
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    https://www.springframework.org/schema/beans/spring-beans.xsd">
```

```
<bean id="bookRepository"
      class="com.library.repository.BookRepository"/>

<bean id="bookService"
      class="com.library.service.BookService">

    <property name="bookRepository"
              ref="bookRepository"/>

</bean>

</beans>
```

Create Repository Class

Package:

com.library.repository

BookRepository.java

```
package com.library.repository;

public class BookRepository {

    public void displayBook() {
        System.out.println("Book Repository: Book data accessed successfully.");
    }
}
```

Create Service Class

Package:

com.library.service

BookService.java

```
package com.library.service;

import com.library.repository.BookRepository;

public class BookService {

    private BookRepository bookRepository;

    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public void showBook() {
        System.out.println("Book Service Called");
        bookRepository.displayBook();
    }
}
```

```
}  
}
```

Create Main Class

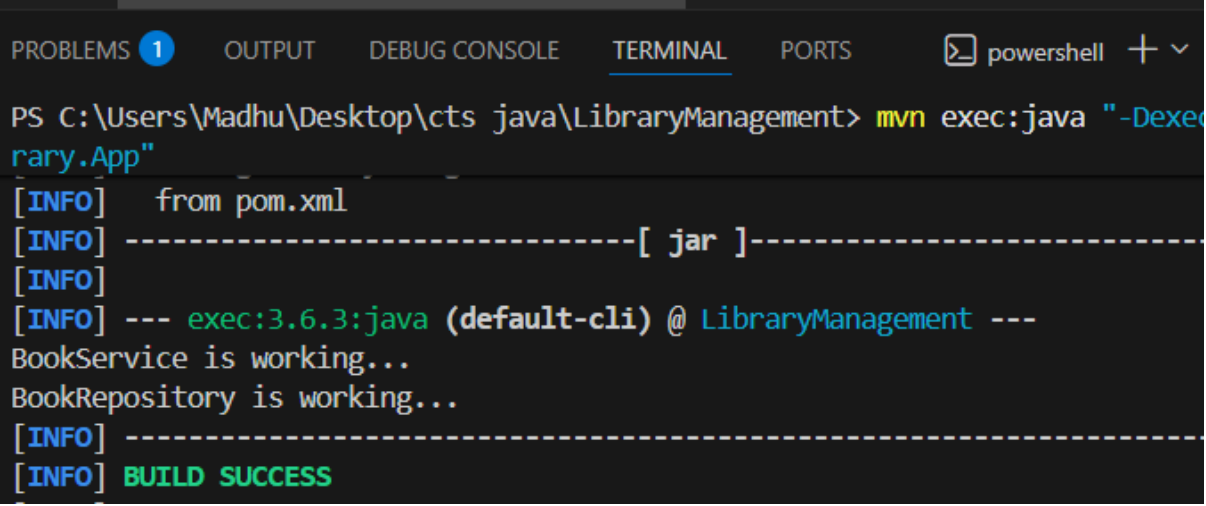
Package:

com.library

LibraryApplication.java

```
package com.library;  
  
import org.springframework.context.ApplicationContext;  
import org.springframework.context.support.ClassPathXmlApplicationContext;  
  
import com.library.service.BookService;  
  
public class LibraryApplication {  
  
    public static void main(String[] args) {  
  
        ApplicationContext context =  
            new ClassPathXmlApplicationContext("applicationContext.xml");  
  
        BookService service =  
            context.getBean("bookService", BookService.class);  
  
        service.showBook();  
  
        ((ClassPathXmlApplicationContext) context).close();  
    }  
}
```

Output



```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell + v  
PS C:\Users\Madhu\Desktop\cts java\LibraryManagement> mvn exec:java "-Dexec.mainClass=com.library.Library.App"  
[INFO] from pom.xml  
[INFO] -----[ jar ]-----  
[INFO] --- exec:3.6.3:java (default-cli) @ LibraryManagement ---  
BookService is working...  
BookRepository is working...  
[INFO] -----  
[INFO] BUILD SUCCESS
```

Exercise 2: Implementing Dependency Injection

BookRepository.java

```
package com.libraryy.repository;

public class BookRepository {

    public void displayRepository() {
        System.out.println("Book Repository is working");
    }
}
```

BookService.java

```
package com.libraryy.service;

import com.libraryy.repository.BookRepository;

public class BookService {

    private BookRepository bookRepository;

    // Setter Injection
    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public void displayService() {
        System.out.println("Book Service is working.");
        bookRepository.displayRepository();
    }
}
```

applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <!-- BookRepository Bean -->
    <bean id="bookRepository"
          class="com.libraryy.repository.BookRepository"/>

    <!-- BookService Bean -->
    <bean id="bookService"
          class="com.libraryy.service.BookService">

        <property name="bookRepository" ref="bookRepository"/>

    </bean>
```

```
</beans>
```

LibraryManagementApplication.java (or Appt.java)

```
package com.libraryy;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

import com.libraryy.service.BookService;

public class Appt {

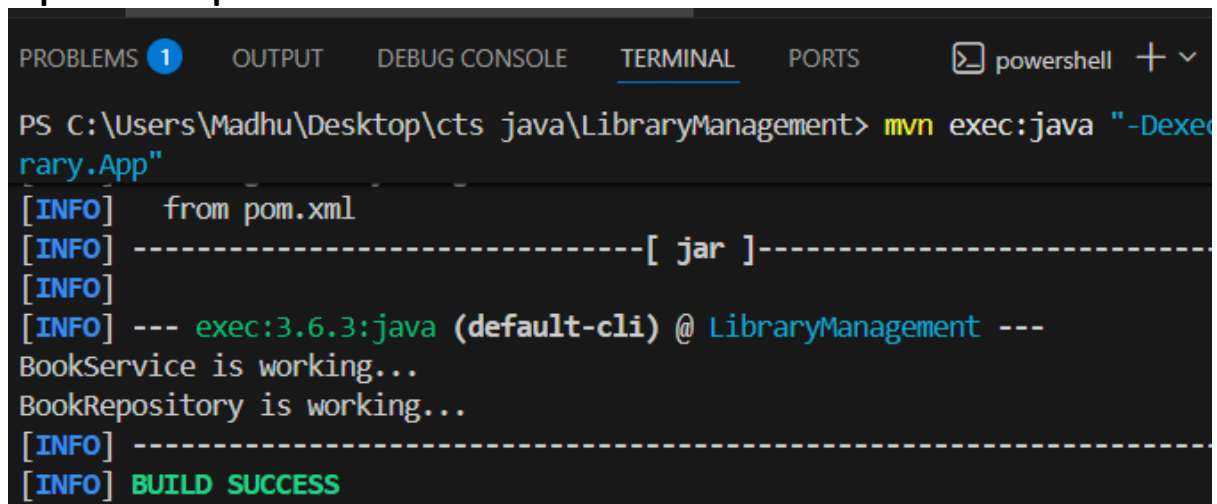
    public static void main(String[] args) {

        ApplicationContext context =
            new ClassPathXmlApplicationContext("applicationContext.xml");

        BookService service =
            (BookService) context.getBean("bookService");

        service.displayService();
    }
}
```

Expected Output



```
PS C:\Users\Madhu\Desktop\cts java\LibraryManagement> mvn exec:java "-Dexec.mainClass=com.libraryy.App"
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- exec:3.6.3:java (default-cli) @ LibraryManagement ---
BookService is working...
BookRepository is working...
[INFO] -----
[INFO] BUILD SUCCESS
```

Exercise 3: Implementing Logging with Spring AOP

1. pom.xml (Add Spring AOP Dependencies)

Add the following dependencies inside the <dependencies> section.

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-aop</artifactId>
  <version>5.3.30</version>
```

```
</dependency>
```

```
<dependency>
  <groupId>org.aspectj</groupId>
  <artifactId>aspectjweaver</artifactId>
  <version>1.9.22.1</version>
</dependency>
```

LoggingAspect.java

Location:

```
src/main/java/com/library/aspect/LoggingAspect.java
package com.library.aspect;
```

```
import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;
```

```
@Aspect
public class LoggingAspect {

    @Around("execution(* com.library.service.*(..))")
    public Object logExecutionTime(ProceedingJoinPoint joinPoint) throws Throwable {

        long start = System.currentTimeMillis();

        Object result = joinPoint.proceed();

        long end = System.currentTimeMillis();

        System.out.println(joinPoint.getSignature().getName()
            + " executed in " + (end - start) + " ms");

        return result;
    }
}
```

applicationContext.xml

Location:

```
src/main/resources/applicationContext.xml
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:aop="http://www.springframework.org/schema/aop"
  xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    https://www.springframework.org/schema/beans/spring-beans.xsd
    http://www.springframework.org/schema/aop
    https://www.springframework.org/schema/aop/spring-aop.xsd">

  <bean id="bookRepository"
```

```

        class="com.library.repository.BookRepository"/>

<bean id="bookService"
    class="com.library.service.BookService">
    <property name="bookRepository" ref="bookRepository"/>
</bean>

<bean id="loggingAspect"
    class="com.library.aspect.LoggingAspect"/>

<aop:aspectj-autoproxy/>

</beans>

```

BookRepository.java

```

package com.library.repository;

public class BookRepository {

    public void displayRepository() {
        System.out.println("BookRepository is working...");
    }
}

```

BookService.java

```

package com.library.service;

import com.library.repository.BookRepository;

public class BookService {

    private BookRepository bookRepository;

    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public void displayService() {
        System.out.println("BookService is working...");
        bookRepository.displayRepository();
    }
}

```

App.java

```

package com.library;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

```

```

import com.library.service.BookService;

public class App {

    public static void main(String[] args) {

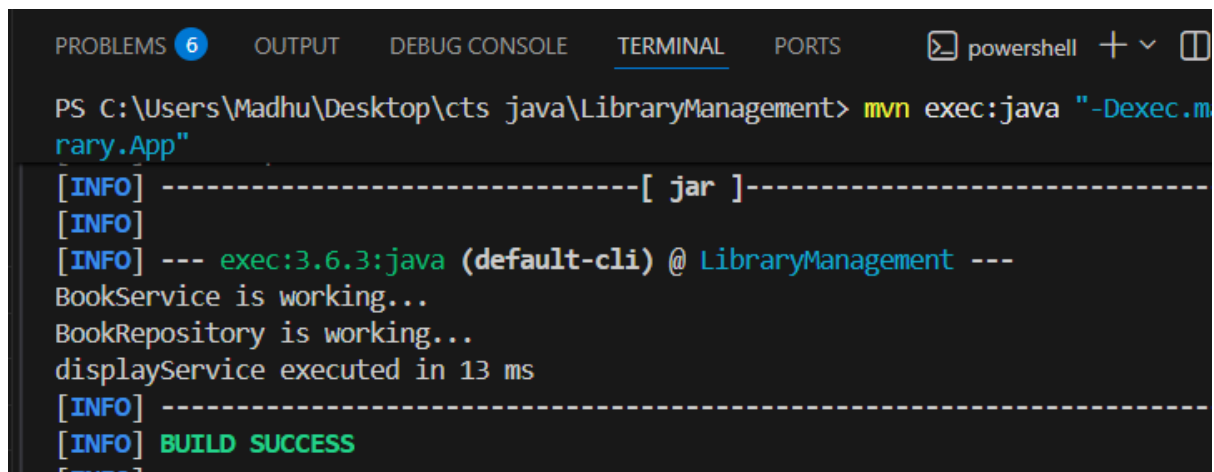
        ApplicationContext context =
            new ClassPathXmlApplicationContext("applicationContext.xml");

        BookService service =
            (BookService) context.getBean("bookService");

        service.displayService();
    }
}

```

Output



```

PS C:\Users\Madhu\Desktop\cts java\LibraryManagement> mvn exec:java -Dexec.mainClass=com.library.App
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- exec:3.6.3:java (default-cli) @ LibraryManagement ---
BookService is working...
BookRepository is working...
displayService executed in 13 ms
[INFO] -----
[INFO] BUILD SUCCESS
[INFO]

```

Exercise 4: Creating and Configuring a Maven Project

Add Spring Dependencies

Open **pom.xml** and add the following dependencies inside the <dependencies> section.

Spring Context

```

<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-context</artifactId>
  <version>5.3.30</version>
</dependency>

```

Spring AOP

```

<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-aop</artifactId>
  <version>5.3.30</version>
</dependency>

```

Spring WebMVC

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-webmvc</artifactId>
  <version>5.3.30</version>
</dependency>
```

Configure Maven Compiler Plugin

Update the **Maven Compiler Plugin** in the <build> section of **pom.xml**.

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <version>3.13.0</version>

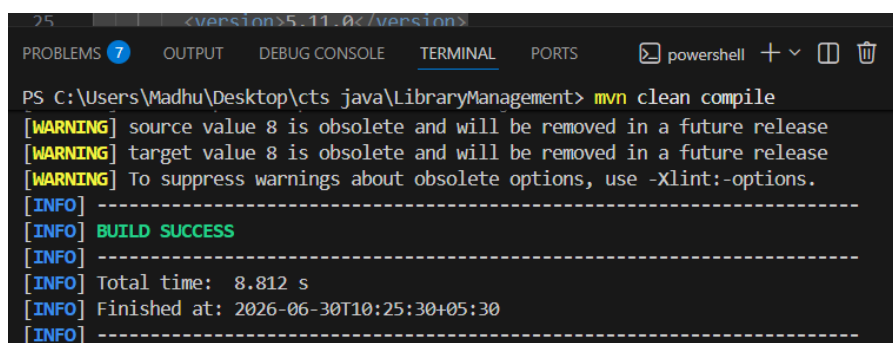
  <configuration>
    <source>1.8</source>
    <target>1.8</target>
  </configuration>
</plugin>
```

Configure Java Version

Update the <properties> section in **pom.xml**.

```
<properties>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <maven.compiler.source>1.8</maven.compiler.source>
  <maven.compiler.target>1.8</maven.compiler.target>
</properties>
```

Result



```
PS C:\Users\Madhu\Desktop\cts java\LibraryManagement> mvn clean compile
[WARNING] source value 8 is obsolete and will be removed in a future release
[WARNING] target value 8 is obsolete and will be removed in a future release
[WARNING] To suppress warnings about obsolete options, use -Xlint:-options.
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 8.812 s
[INFO] Finished at: 2026-06-30T10:25:30+05:30
[INFO] -----
```

Exercise 5: Configuring the Spring IoC Container

applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>
```

```

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:aop="http://www.springframework.org/schema/aop"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           https://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.springframework.org/schema/aop
           https://www.springframework.org/schema/aop/spring-aop.xsd">

    <bean id="bookRepository"
          class="com.library.repository.BookRepository"/>

    <bean id="bookService"
          class="com.library.service.BookService">

        <property name="bookRepository"
                  ref="bookRepository"/>

    </bean>

    <bean id="loggingAspect"
          class="com.library.aspect.LoggingAspect"/>

    <aop:aspectj-autoproxy/>

</beans>

```

Create BookRepository Class

Location:

```

src/main/java/com/library/repository/BookRepository.java
package com.library.repository;

```

```

public class BookRepository {

    public void displayRepository() {
        System.out.println("BookRepository is working...");
    }

}

```

Create BookService Class

Location:

```

src/main/java/com/library/service/BookService.java
package com.library.service;

```

```

import com.library.repository.BookRepository;

public class BookService {

    private BookRepository bookRepository;

    public void setBookRepository(BookRepository bookRepository) {

```

```

        this.bookRepository = bookRepository;
    }

    public void displayService() {
        System.out.println("BookService is working...");
        bookRepository.displayRepository();
    }
}

```

Step 4: Create Main Class

Location:

```

src/main/java/com/library/App.java
package com.library;

```

```

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

import com.library.service.BookService;

public class App {

    public static void main(String[] args) {

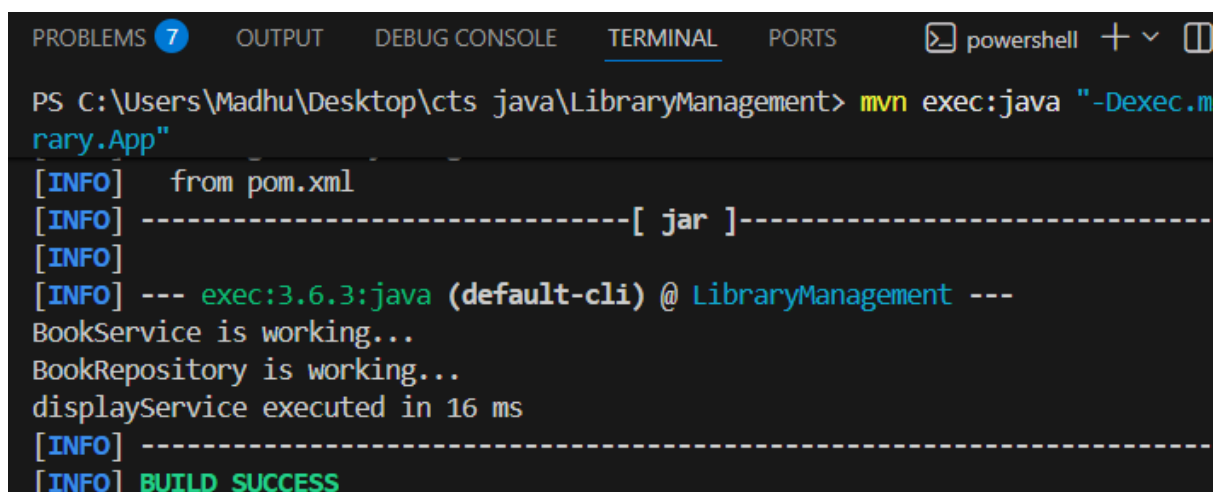
        ApplicationContext context =
            new ClassPathXmlApplicationContext("applicationContext.xml");

        BookService service =
            (BookService) context.getBean("bookService");

        service.displayService();
    }
}

```

Output



```

PROBLEMS 7 OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell + v
PS C:\Users\Madhu\Desktop\cts java\LibraryManagement> mvn exec:java "-Dexec.m
rary.App"
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- exec:3.6.3:java (default-cli) @ LibraryManagement ---
BookService is working...
BookRepository is working...
displayService executed in 16 ms
[INFO] -----
[INFO] BUILD SUCCESS

```

Exercise 6: Configuring Beans with Annotations

applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xmlns:aop="http://www.springframework.org/schema/aop"
       xsi:schemaLocation="
         http://www.springframework.org/schema/beans
         https://www.springframework.org/schema/beans/spring-beans.xsd
         http://www.springframework.org/schema/context
         https://www.springframework.org/schema/context/spring-context.xsd
         http://www.springframework.org/schema/aop
         https://www.springframework.org/schema/aop/spring-aop.xsd">

    <context:component-scan base-package="com.library"/>

    <bean id="loggingAspect"
          class="com.library.aspect.LoggingAspect"/>

    <aop:aspectj-autoproxy/>

</beans>
```

Configure BookRepository Using @Repository

Location:

```
src/main/java/com/library/repository/BookRepository.java
package com.library.repository;
```

```
import org.springframework.stereotype.Repository;
```

```
@Repository
```

```
public class BookRepository {
```

```
    public void displayRepository() {
        System.out.println("BookRepository is working...");
    }
}
```

Configure BookService Using @Service

Location:

```
src/main/java/com/library/service/BookService.java
package com.library.service;
```

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
```

```
import com.library.repository.BookRepository;
```

```

@Service
public class BookService {

    @Autowired
    private BookRepository bookRepository;

    public void displayService() {
        System.out.println("BookService is working...");
        bookRepository.displayRepository();
    }
}

```

Step 4: Create the Main Class

Location:

```

src/main/java/com/library/App.java
package com.library;

```

```

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

import com.library.service.BookService;

public class App {

    public static void main(String[] args) {

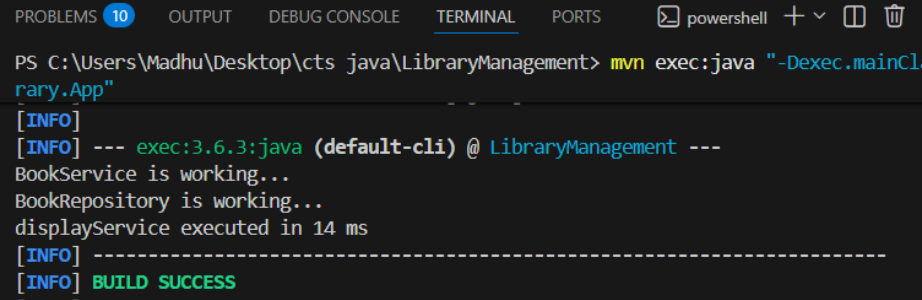
        ApplicationContext context =
            new ClassPathXmlApplicationContext("applicationContext.xml");

        BookService service = context.getBean(BookService.class);

        service.displayService();
    }
}

```

Result



```

PROBLEMS 10 OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell + - [ ]
PS C:\Users\Madhu\Desktop\cts_java\LibraryManagement> mvn exec:java -Dexec.mainClass=com.library.App
[INFO]
[INFO] --- exec:3.6.3:java (default-cli) @ LibraryManagement ---
BookService is working...
BookRepository is working...
displayService executed in 14 ms
[INFO] -----
[INFO] BUILD SUCCESS
[INFO]

```

Exercise 7: Implementing Constructor and Setter Injection

applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
         http://www.springframework.org/schema/beans
         https://www.springframework.org/schema/beans/spring-beans.xsd">

    <!-- BookRepository Bean -->
    <bean id="bookRepository"
          class="com.library.repository.BookRepository"/>

    <!-- BookService Bean -->
    <bean id="bookService"
          class="com.library.service.BookService">

        <!-- Constructor Injection -->
        <constructor-arg ref="bookRepository"/>

        <!-- Setter Injection -->
        <property name="bookRepository" ref="bookRepository"/>

    </bean>

</beans>
```

Create BookRepository Class

Location:

```
src/main/java/com/library/repository/BookRepository.java
package com.library.repository;
```

```
public class BookRepository {

    public void displayRepository() {
        System.out.println("BookRepository is working...");
    }
}
```

Create BookService Class

Location:

```
src/main/java/com/library/service/BookService.java
package com.library.service;
```

```
import com.library.repository.BookRepository;
```

```

public class BookService {

    private BookRepository bookRepository;

    // Constructor Injection
    public BookService(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
        System.out.println("Constructor Injection Executed");
    }

    // Setter Injection
    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
        System.out.println("Setter Injection Executed");
    }

    public void displayService() {
        System.out.println("BookService is working...");
        bookRepository.displayRepository();
    }
}

```

Create Main Class

Location:

```

src/main/java/com/library/App.java
package com.library;

```

```

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

```

```

import com.library.service.BookService;

```

```

public class App {

    public static void main(String[] args) {

        ApplicationContext context =
            new ClassPathXmlApplicationContext("applicationContext.xml");

        BookService service =
            context.getBean("bookService", BookService.class);

        service.displayService();
    }
}

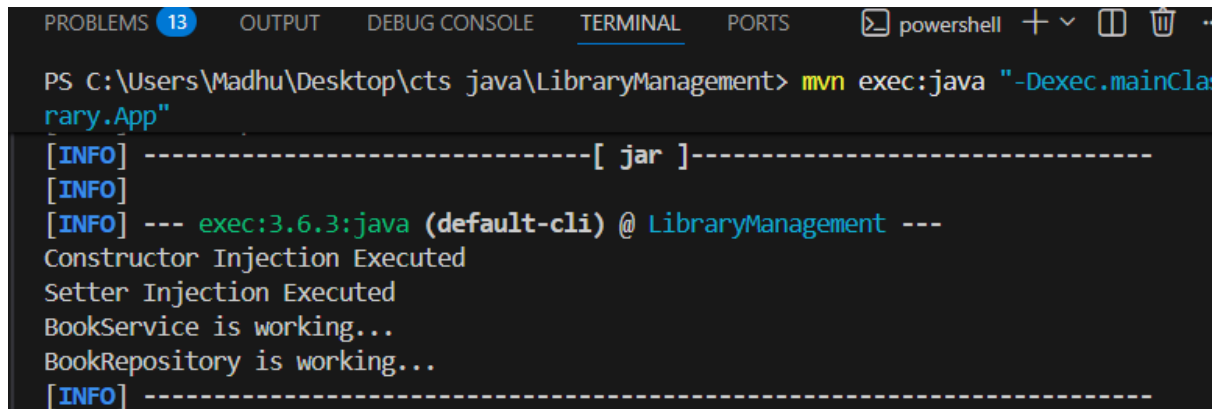
```

```

Constructor Injection Executed
Setter Injection Executed
BookService is working...
BookRepository is working...

```

Result



```
PS C:\Users\Madhu\Desktop\cts java\LibraryManagement> mvn exec:java "-Dexec.mainClass=Library.App"
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- exec:3.6.3:java (default-cli) @ LibraryManagement ---
Constructor Injection Executed
Setter Injection Executed
BookService is working...
BookRepository is working...
[INFO] -----
```

Exercise 8: Implementing Basic AOP with Spring

Add the following dependencies in the pom.xml file.

Spring AOP

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-aop</artifactId>
  <version>5.3.30</version>
</dependency>
```

AspectJ Weaver

```
<dependency>
  <groupId>org.aspectj</groupId>
  <artifactId>aspectjweaver</artifactId>
  <version>1.9.22.1</version>
</dependency>
```

Create LoggingAspect Class

Location:

```
src/main/java/com/library/aspect/LoggingAspect.java
package com.library.aspect;
```

```
import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.After;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
```

```
@Aspect
```

```
public class LoggingAspect {
```

```
    @Before("execution(* com.library.service.*(..))")
    public void beforeMethod(JoinPoint joinPoint) {
        System.out.println("Before executing : "
            + joinPoint.getSignature().getName());
    }
}
```

```

    }

    @After("execution(* com.library.service.*.*(..))")
    public void afterMethod(JoinPoint joinPoint) {
        System.out.println("After executing : "
            + joinPoint.getSignature().getName());
    }
}

```

Configure the Aspect

Update the applicationContext.xml file.

```

<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:aop="http://www.springframework.org/schema/aop"
    xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        https://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/aop
        https://www.springframework.org/schema/aop/spring-aop.xsd">

    <bean id="bookRepository"
        class="com.library.repository.BookRepository"/>

    <bean id="bookService"
        class="com.library.service.BookService">

        <property name="bookRepository" ref="bookRepository"/>

    </bean>

    <bean id="loggingAspect"
        class="com.library.aspect.LoggingAspect"/>

    <aop:aspectj-autoproxy/>

</beans>

```

Create BookRepository Class

Location:

src/main/java/com/library/repository/BookRepository.java
 package com.library.repository;

```

public class BookRepository {

    public void displayRepository() {
        System.out.println("BookRepository is working...");
    }
}

```

Create BookService Class

Location:

```
src/main/java/com/library/service/BookService.java
package com.library.service;

import com.library.repository.BookRepository;

public class BookService {

    private BookRepository bookRepository;

    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public void displayService() {
        System.out.println("BookService is working...");
        bookRepository.displayRepository();
    }
}
```

Create Main Class

Location:

```
src/main/java/com/library/App.java
package com.library;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

import com.library.service.BookService;

public class App {

    public static void main(String[] args) {

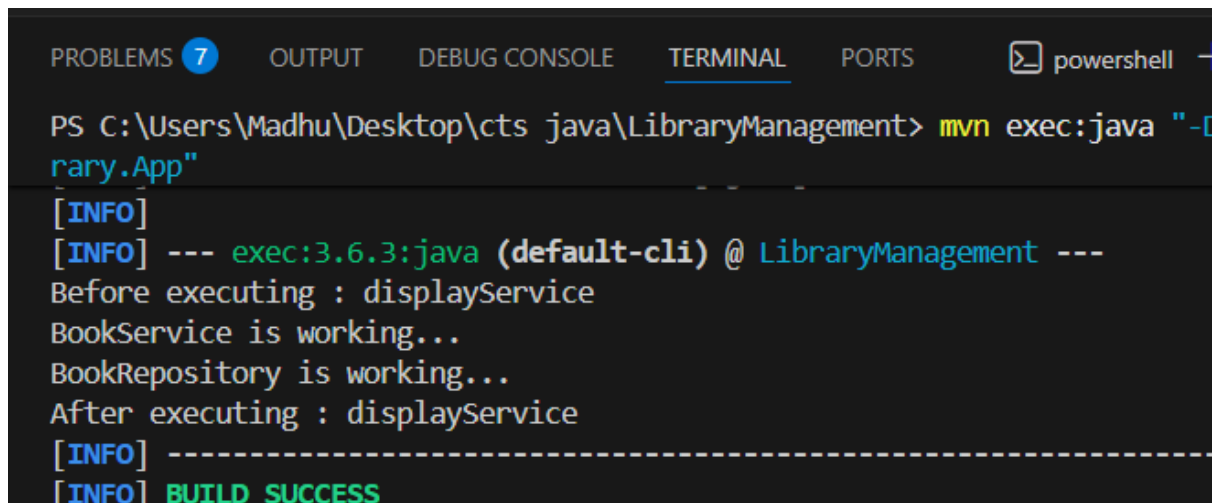
        ApplicationContext context =
            new ClassPathXmlApplicationContext("applicationContext.xml");

        BookService service =
            context.getBean("bookService", BookService.class);

        service.displayService();
    }
}

mvn clean compile
```

Result



```
PROBLEMS 7 OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell
PS C:\Users\Madhu\Desktop\cts java\LibraryManagement> mvn exec:java "-D
rary.App"
[INFO]
[INFO] --- exec:3.6.3:java (default-cli) @ LibraryManagement ---
Before executing : displayService
BookService is working...
BookRepository is working...
After executing : displayService
[INFO] -----
[INFO] BUILD SUCCESS
```

Exercise 9: Creating a Spring Boot Application

Configure application.properties

Location:

```
src/main/resources/application.properties
spring.application.name=LibraryManagement

spring.datasource.url=jdbc:h2:mem:librarydb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update

spring.h2.console.enabled=true

spring.jpa.show-sql=true
```

Create the Book Entity

Location:

```
src/main/java/com/library/entity/Book.java
package com.library.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
```

```
@Entity
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;
    private String author;

    public Book() {
    }

    public Book(Long id, String title, String author) {
        this.id = id;
        this.title = title;
        this.author = author;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getAuthor() {
        return author;
    }

    public void setAuthor(String author) {
        this.author = author;
    }
}
```

Create the Repository Interface

Location:

```
src/main/java/com/library/repository/BookRepository.java
package com.library.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import com.library.entity.Book;
```

```
@Repository
public interface BookRepository extends JpaRepository<Book, Long> {

}
```

Create the REST Controller

Location:

```
src/main/java/com/library/controller/BookController.java
package com.library.controller;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import com.library.entity.Book;
import com.library.repository.BookRepository;

@RestController
@RequestMapping("/books")
public class BookController {

    @Autowired
    private BookRepository bookRepository;

    @GetMapping
    public List<Book> getAllBooks() {
        return bookRepository.findAll();
    }

    @PostMapping
    public Book addBook(@RequestBody Book book) {
        return bookRepository.save(book);
    }

    @PutMapping("/{id}")
    public Book updateBook(@PathVariable Long id, @RequestBody Book book) {
        book.setId(id);
        return bookRepository.save(book);
    }

    @DeleteMapping("/{id}")
    public String deleteBook(@PathVariable Long id) {
        bookRepository.deleteById(id);
        return "Book deleted successfully!";
    }
}
```

Main Application Class

Location:

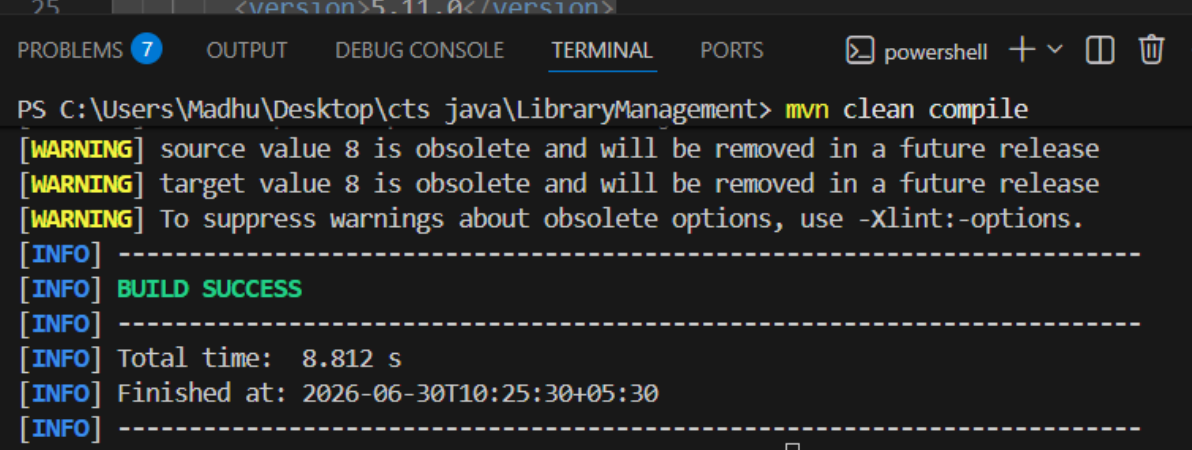
```
src/main/java/com/library/LibraryManagementApplication.java
package com.library;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class LibraryManagementApplication {

    public static void main(String[] args) {
        SpringApplication.run(LibraryManagementApplication.class, args);
    }
}
```

Result:



The screenshot shows a terminal window with the following content:

```
PS C:\Users\Madhu\Desktop\cts java\LibraryManagement> mvn clean compile
[WARNING] source value 8 is obsolete and will be removed in a future release
[WARNING] target value 8 is obsolete and will be removed in a future release
[WARNING] To suppress warnings about obsolete options, use -Xlint:-options.
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 8.812 s
[INFO] Finished at: 2026-06-30T10:25:30+05:30
[INFO] -----
```